COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 20, 2026	**DATE SUBMITTED:**	October 22, 2026
NAME:	De Leon, Benj Federic C. Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 ALGORITHM COMPLEXITY

1. PROCEDURE OUTPUT

Output:

Code

Merge Sort

```cpp
#include <iostream>
#include <iomanip>
#include "sorting.h"

int main() {
    std::cout << std::fixed << std::setprecision(3);
    std::cout << "Best and Worst Case: Insertion Sort vs Merge Sort\n";
    std::cout << "Best case = sorted input, Worst case = reverse-sorted input\n\n";
    std::cout << std::left << std::setw(8) << "n"
              << std::setw(12) << "Case"
              << std::setw(24) << "Insertion Sort (us)"
              << std::setw(24) << "Merge Sort (us)" << "\n";
    std::cout << std::string(68, '-') << "\n";

    for (int n : {100, 1000, 10000}) {
        int reps = (n == 10000) ? 20 : (n == 1000 ? 200 : 5000);
        std::vector<int> best = makeBestCase(n);
        std::vector<int> worst = makeWorstCase(n);

        std::cout << std::left << std::setw(8) << n << std::setw(12) << "Best"
                  << std::setw(24) << timeSortUs(insertionSort, best, reps)
                  << std::setw(24) << timeSortUs(mergeSort, best, reps) << "\n";
        std::cout << std::left << std::setw(8) << n << std::setw(12) << "Worst"
                  << std::setw(24) << timeSortUs(insertionSort, worst, reps)
                  << std::setw(24) << timeSortUs(mergeSort, worst, reps) << "\n";
    }

    std::cout << "\nTheoretical time complexity\n";
    std::cout << "Insertion Sort: best O(n), worst O(n^2)\n";
    std::cout << "Merge Sort:     best O(n log n), worst O(n log n)\n";
    return 0;
}
```

```cpp
#ifndef SORTING_H
#define SORTING_H

#include <vector>
#include <chrono>
#include <algorithm>


//Algorithm 2: Merge Sort
inline void merge(std::vector<int> &a, int l, int m, int r) {
    std::vector<int> tmp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) tmp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) tmp.push_back(a[i++]);
    while (j <= r) tmp.push_back(a[j++]);
    for (int k = 0; k < (int)tmp.size(); k++) a[l + k] = tmp[k];
}

inline void mergeSort(std::vector<int> &a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}


inline std::vector<int> makeBestCase(int n) {      // already sorted
    std::vector<int> v(n);
    for (int i = 0; i < n; i++) v[i] = i;
    return v;
}

inline std::vector<int> makeWorstCase(int n) {     // reverse sorted
    std::vector<int> v(n);
    for (int i = 0; i < n; i++) v[i] = n - i;
    return v;
}

//Average time (microseconds)
inline double timeSortUs(void (*sortFn)(std::vector<int> &, int, int),
                         const std::vector<int> &data, int reps) {
    double total = 0;
    for (int r = 0; r < reps; r++) {
        std::vector<int> c = data;
        auto t0 = std::chrono::steady_clock::now();
        sortFn(c, 0, (int)c.size() - 1);
        auto t1 = std::chrono::steady_clock::now();
        total += std::chrono::duration<double, std::micro>(t1 - t0).count();
    }
    return total / reps;
}

#endif
```

Insertion Sort

```cpp
#include <iostream>
#include <iomanip>
#include "sorting.h"

int main() {
    std::cout << std::fixed << std::setprecision(3);
    std::cout << "Best and Worst Case: Insertion Sort vs Merge Sort\n";
    std::cout << "Best case = sorted input, Worst case = reverse-sorted input\n\n";
    std::cout << std::left << std::setw(8) << "n"
              << std::setw(12) << "Case"
              << std::setw(24) << "Insertion Sort (us)"
              << std::setw(24) << "Merge Sort (us)" << "\n";
    std::cout << std::string(68, '-') << "\n";

    for (int n : {100, 1000, 10000}) {
        int reps = (n == 10000) ? 20 : (n == 1000 ? 200 : 5000);
        std::vector<int> best = makeBestCase(n);
        std::vector<int> worst = makeWorstCase(n);

        std::cout << std::left << std::setw(8) << n << std::setw(12) << "Best"
                  << std::setw(24) << timeSortUs(insertionSort, best, reps)
                  << std::setw(24) << timeSortUs(mergeSort, best, reps) << "\n";
        std::cout << std::left << std::setw(8) << n << std::setw(12) << "Worst"
                  << std::setw(24) << timeSortUs(insertionSort, worst, reps)
                  << std::setw(24) << timeSortUs(mergeSort, worst, reps) << "\n";
    }

    std::cout << "\nTheoretical time complexity\n";
    std::cout << "Insertion Sort: best O(n), worst O(n^2)\n";
    std::cout << "Merge Sort:     best O(n log n), worst O(n log n)\n";
    return 0;
}
```

```cpp
#ifndef SORTING_H
#define SORTING_H

#include <vector>
#include <chrono>
#include <algorithm>

//Algorithm 1: Insertion Sort
inline void insertionSort(std::vector<int> &a, int l, int r) {
    for (int i = l + 1; i <= r; i++) {
        int key = a[i], j = i - 1;
        while (j >= l && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}



inline void mergeSort(std::vector<int> &a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}


inline std::vector<int> makeBestCase(int n) {      // already sorted
    std::vector<int> v(n);
    for (int i = 0; i < n; i++) v[i] = i;
    return v;
}

inline std::vector<int> makeWorstCase(int n) {     // reverse sorted
    std::vector<int> v(n);
    for (int i = 0; i < n; i++) v[i] = n - i;
    return v;
}

//Average time (microseconds)
inline double timeSortUs(void (*sortFn)(std::vector<int> &, int, int),
                         const std::vector<int> &data, int reps) {
    double total = 0;
    for (int r = 0; r < reps; r++) {
        std::vector<int> c = data;
        auto t0 = std::chrono::steady_clock::now();
        sortFn(c, 0, (int)c.size() - 1);
        auto t1 = std::chrono::steady_clock::now();
        total += std::chrono::duration<double, std::micro>(t1 - t0).count();
    }
    return total / reps;
}

#endif
```

Output

```
Best and Worst Case: Insertion Sort vs Merge Sort
Best case = sorted input, Worst case = reverse-sorted input

n       Case        Insertion Sort (us)     Merge Sort (us)
-----------------------------------------------------------------
100     Best        0.812                   85.917
100     Worst       33.527                  79.146
1000    Best        7.806                   853.790
1000    Worst       2828.097                701.579
10000   Best        49.050                  7889.530
10000   Worst       226209.110              7037.805

Theoretical time complexity
Insertion Sort: best O(n), worst O(n^2)
Merge Sort:     best O(n log n), worst O(n log n)
```

	Best Case	Worst Case	Analysis
Merge Sort	O(n)	O(n^2)	Insertion Sort is efficient when the data is already sorted. It becomes slower when the data is in reverse order because it performs more shifts and comparisons.
Insertion Sort	O(n log n)	O(n log n)	Merge Sort divides the array into smaller parts and merges them in sorted order. Its time complexity remains $O(n \log n)$ regardless of the initial order of the data.

Table 14-1: Best- and Worst-Case Analysis using Theoretical Tools

Analysis		
	De Leon	Insertion Sort is O(n) in the best case (sorted data) and O(n²) in the worst case (reverse order). Merge Sort is O(n log n) in both cases because it always splits and merges the array.
	Mendoza	Insertion Sort has a best-case complexity of $O(n)$ and a worst-case complexity of $O(n^2)$. Merge Sort has $O(n \log n)$ complexity in both cases, making it more efficient for large datasets.

n	Case	Insertion Sort (us)	Merge Sort (us)
100	Best	0.812	85.197
100	Worst	33.527	79.146
1000	Best	7.806	835.790
1000	Worst	2828.097	701.579
10000	Best	49.050	7889.530
10000	Worst	226209.110	7037.805

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

Analysis		
	De Leon	Insertion Sort is O(n) on sorted data but O(n²) on reversed data (226,209 us at n = 10,000). Merge Sort is O(n log n) for both cases (about 7,000 us), so it is much faster on large inputs.
	Mendoza	Insertion Sort is faster when the data is already sorted, but becomes slower when the data is in reverse order. Merge Sort has more consistent performance and is faster for large datasets in the worst case.

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A:

Code	
Linear Search	

```cpp
#ifndef SEARCHING_H
#define SEARCHING_H

#include <vector>
#include <chrono>
#include <algorithm>

// Linear Search (Worst Case: element not found or at the last index)
inline int linearSearch(const std::vector<int>& arr, int target) {
    for (size_t i = 0; i < arr.size(); ++i) {
        if (arr[i] == target) {
            return static_cast<int>(i);
        }
    }
    return -1;
}



inline std::vector<int> generateSortedArray(int n) {
    std::vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        v[i] = i;
    }
    return v;
}

inline double timeSearchUs(int (*searchFn)(const std::vector<int>&, int),
                           const std::vector<int>& data, int target, int reps) {
    double total = 0;
    for (int r = 0; r < reps; ++r) {
        auto t0 = std::chrono::steady_clock::now();
        volatile int result = searchFn(data, target); // volatile prevents compiler optimization
        (void)result;
        auto t1 = std::chrono::steady_clock::now();
        total += std::chrono::duration<double, std::micro>(t1 - t0).count();
    }
    return total / reps;
}

#endif
```

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include "suppSorting.h"

int main() {
    std::cout << std::fixed << std::setprecision(5);
    std::cout << "Empirical Complexity Benchmark: Linear Search vs Binary Search\n";
    std::cout << "Worst-Case Analysis (Target element absent from array)\n\n";

    std::cout << std::left << std::setw(12) << "N"
              << std::setw(30) << "Binary Search (us)"
              << std::setw(30) << "Linear Search (us)" << "\n";
    std::cout << std::string(72, '-') << "\n";

    for (int n : {1000, 10000, 100000}) {
        int reps = (n == 100000) ? 500 : ((n == 10000) ? 2000 : 10000);

        std::vector<int> data = generateSortedArray(n);
        int target = -1;

        double binaryTime = timeSearchUs(binarySearch, data, target, reps);
        double linearTime = timeSearchUs(linearSearch, data, target, reps);

        std::cout << std::left << std::setw(12) << n
                  << std::setw(30) << binaryTime
                  << std::setw(30) << linearTime << "\n";
    }

    std::cout << "\nTheoretical Time Complexity:\n";
    std::cout << "Binary Search: Worst-case O(log N), Best-case O(1), Space O(1)\n";
    std::cout << "Linear Search: Worst-case O(N),     Best-case O(1), Space O(1)\n";

    return 0;
}
```

Binary Search



```cpp
#ifndef SEARCHING_H
#define SEARCHING_H

#include <vector>
#include <chrono>
#include <algorithm>


// Binary Search (Iterative - requires sorted input)
inline int binarySearch(const std::vector<int>& arr, int target) {
    int low = 0;
    int high = static_cast<int>(arr.size()) - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

inline std::vector<int> generateSortedArray(int n) {
    std::vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        v[i] = i;
    }
    return v;
}

inline double timeSearchUs(int (*searchFn)(const std::vector<int>&, int),
                           const std::vector<int>& data, int target, int reps) {
    double total = 0;
    for (int r = 0; r < reps; ++r) {
        auto t0 = std::chrono::steady_clock::now();
        volatile int result = searchFn(data, target); // volatile prevents compiler optimization
        (void)result;
        auto t1 = std::chrono::steady_clock::now();
        total += std::chrono::duration<double, std::micro>(t1 - t0).count();
    }
    return total / reps;
}

#endif
```

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include "suppSorting.h"

int main() {
    std::cout << std::fixed << std::setprecision(5);
    std::cout << "Empirical Complexity Benchmark: Linear Search vs Binary Search\n";
    std::cout << "Worst-Case Analysis (Target element absent from array)\n\n";

    std::cout << std::left << std::setw(12) << "N"
              << std::setw(30) << "Binary Search (us)"
              << std::setw(30) << "Linear Search (us)" << "\n";
    std::cout << std::string(72, '-') << "\n";

    for (int n : {1000, 10000, 100000}) {
        int reps = (n == 100000) ? 500 : ((n == 10000) ? 2000 : 10000);

        std::vector<int> data = generateSortedArray(n);
        int target = -1;

        double binaryTime = timeSearchUs(binarySearch, data, target, reps);
        double linearTime = timeSearchUs(linearSearch, data, target, reps);

        std::cout << std::left << std::setw(12) << n
                  << std::setw(30) << binaryTime
                  << std::setw(30) << linearTime << "\n";
    }

    std::cout << "\nTheoretical Time Complexity:\n";
    std::cout << "Binary Search: Worst-case O(log N), Best-case O(1), Space O(1)\n";
    std::cout << "Linear Search: Worst-case O(N),     Best-case O(1), Space O(1)\n";

    return 0;
}
```

Output

```
N           Binary Search (us)            Linear Search (us)
------------------------------------------------------------------------
1000        0.12232                       5.65611
10000       0.15945                       55.96860
100000      0.18220                       562.36340

Theoretical Time Complexity:
Binary Search: Worst-case O(log N), Best-case O(1), Space O(1)
Linear Search: Worst-case O(N),     Best-case O(1), Space O(1)
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA14p1>
```

Theoretical analysis:

Linear	Binary
Algorithm LinearSearch(Array, Size, Target) 1. Set index i = 0 2. While i < Size do: a. If Array[i] == Target then: i. Return i // Target found b. Set i = i + 1 3. Return -1 // Target not found	Algorithm BinarySearch(Array, Size, Target) 1. Set low = 0 2. Set high = Size - 1 3. While low <= high do: a. Set mid = low + (high - low) / 2 b. If Array[mid] == Target then: i. Return mid // Target found c. Else if Array[mid] < Target then: i. Set low = mid + 1 d. Else: i. Set high = mid - 1 4. Return -1 // Target not found

Empirical analysis:

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.12232 us	5.65611 us
10,000	0.15945 us	55.96860 us
100,000	0.18220 us	562.36340 us

Analysis	
De Leon	Linear search time increases directly as data grows because it scans every item. Binary search stays extremely fast even with large inputs because it splits the search range in half each step, though it requires sorted data.
Mendoza	Linear Search becomes slower as the amount of data increases because it checks each item one by one. Binary Search remains faster for large datasets because it cuts the search area in half each step, but the data must be sorted first.

Conclusion	
De Leon	Algorithm complexity shows how speed changes as data grows. Testing proved Insertion Sort gets slow on big reversed lists, while Merge Sort stays fast. Linear Search slows down as data grows, but Binary Search stays fast on sorted data. Always pick the right algorithm for your data size.
Mendoza	Algorithm complexity explains how an algorithm's performance changes as the data size increases. Testing showed that Insertion Sort becomes slower with large, reversed data, while Merge Sort maintains better performance. Linear Search also slows down as data grows, whereas Binary Search remains fast when the data is sorted. Therefore, choosing the right algorithm depends on the size and arrangement of the data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, explanation of concepts, and drafting analysis and conclusion text.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Claude was used to help write and test the C++ code and to draft the missing analysis, answers and conclusions. We reviewed and ran the code and edited the text before submitting

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.